

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN

LAB 2
DECISION TREE MODELING
AND IMPROVEMENT

Nhóm 7

Sinh viên

Lưu Huy Minh Quang	–	23127016
Ngô Quang Thắng	–	23127473
Vũ Lê Trọng Văn	–	20127095
Nguyễn Duy Khang	–	23127202

Giảng viên

Bùi Duy Đăng – Bùi Tiến Lên – Võ Nhật Tân

Giới thiệu nhóm

Nhóm thực hiện: Nhóm 7

Phân bổ đóng góp do nhóm thống nhất sau khi đối chiếu GitHub/Trello.

Thành viên	Mô tả công việc	Đối chiếu	Tỷ lệ
Lưu Huy Minh Quang	Khởi tạo và điều phối project; dữ liệu/EDA; baseline; benchmark; cross-dataset; cải tiến mô hình; tổng hợp kết quả; report/slide; custom DT; kiểm thử end-to-end; tích hợp và đóng gói	GitHub/Trello	90,0%
Ngô Quang Thắng	Gini-Entropy; depth sweep; pre/post-pruning; decision rules; feature importance và hình cây	GitHub/Trello	10,0%
Vũ Lê Trọng Văn	Chưa có phần việc hoàn thành độc lập được xác minh	GitHub/Trello	0,0%
Nguyễn Duy Khang	Chưa có phần việc hoàn thành độc lập được xác minh	GitHub/Trello	0,0%

Tỷ lệ đóng góp là mức phân bổ được nhóm thống nhất theo khối lượng deliverable và phần việc có thể đối chiếu trên GitHub/Trello. Bảng phản ánh vai trò thực tế trong quá trình thực hiện, tích hợp và hoàn thiện bản nộp. Nếu có đóng góp offline chưa được ghi nhận, nhóm cần rà soát lại trước khi nộp.

Tóm tắt

Nghiên cứu này dùng Covertypes làm case study chính để xây dựng, phân tích và cải tiến Decision Tree theo đầy đủ yêu cầu Lab. Letter Recognition và Handwritten Digits đóng vai trò robustness datasets; Random Forest, Support Vector Machine và K-Nearest Neighbors là các mô hình đối chứng mở rộng. Mọi thí nghiệm dùng phép chia 80/20 phân tầng với `random_state=42`; lựa chọn tham số chỉ truy cập tập train. Bên cạnh accuracy và macro-F1, nghiên cứu báo cáo error rate, recall theo lớp, train–test gap, thời gian thực thi, depth và number of leaves.

Decision Tree cơ sở trên Covertypes đạt accuracy 93.89%, error rate 6.11% và macro-F1 90.34%, nhưng có train accuracy 100%, depth 41 và 23,956 lá. Nghiên cứu đánh giá ba chiến lược cải tiến: pre-pruning, Cost-Complexity Pruning (CCP) và Hierarchical Shrinkage (HS), cùng một thí nghiệm kết hợp CCP + HS. Cấu hình kết hợp đạt macro-F1 90.53%, giảm train–test gap 17.6% và giảm số lá 31.7% so với baseline. Random Forest dẫn đầu Letter, SVM dẫn đầu Digits, còn Decision Tree dẫn đầu Covertypes trong các cấu hình đã thử nghiệm. Kết quả cho thấy không có mô hình hoặc regularization strategy nào thắng trên mọi chế độ dữ liệu; giá trị rõ nhất của CCP + HS là trade-off giữa chất lượng dự đoán, generalization và complexity trên cây lớn.

Từ khóa: Decision Tree, Hierarchical Shrinkage, Cost-Complexity Pruning, phân loại đa lớp, GPU, Covertypes.

Mục lục

Giới thiệu nhóm	i
Tóm tắt	ii
Danh mục hình	v
Danh mục bảng	vi
1 Giới thiệu	1
1.1 Bối cảnh và động lực	1
1.2 Phạm vi cốt lõi và phần mở rộng	1
1.3 Công trình liên quan	1
1.4 Mục tiêu nghiên cứu	2
1.5 Câu hỏi nghiên cứu	2
1.6 Cấu trúc báo cáo	2
2 Mô tả dữ liệu	3
2.1 Thiết kế phạm vi: một case study chính và hai robustness datasets	3
2.2 Letter Recognition	3
2.3 Handwritten Digits	3
2.4 Covertypes – case study chính	4
2.5 Chuẩn bị dữ liệu và chống leakage	6
3 Mô hình Decision Tree cơ sở	7
3.1 CART xây dựng cây như thế nào	7
3.2 Cấu hình baseline E0	7
3.3 Protocol train, validation và test	8
3.4 Chỉ số đánh giá	8
4 Phân tích cây Decision Tree sinh ra	9
4.1 Cây Covertypes cơ sở	9
4.2 Root split và các luật quyết định tiêu biểu	10
4.3 Hiệu năng baseline và error rate	10
4.4 Confusion matrix và lỗi theo lớp	11
4.5 Feature importance và giới hạn diễn giải	13

5	Cải tiến Decision Tree	14
5.1	Ba chiến lược cải tiến	14
5.2	Chọn tham số chỉ trên tập train	15
5.3	Master comparison E0–E4 trên Covertypes	15
5.4	Vì sao chọn E4 thay vì E3	15
6	So sánh và thảo luận	18
6.1	Cải tiến có khái quát qua ba bộ dữ liệu không?	18
6.2	Benchmark mở rộng: Decision Tree so với ba họ mô hình	18
6.3	Tính tái lập và công bằng thực nghiệm	19
6.4	Môi trường tính toán và diễn giải runtime	19
6.5	Threats to validity	19
6.6	Decision Tree tự cài đặt và đối chiếu với sklearn	20
7	Kết luận	22
7.1	Trả lời các câu hỏi nghiên cứu	22
7.2	Bài học chính	22
7.3	Hướng phát triển	22
	Tài liệu tham khảo	24
A	Bảng benchmark đầy đủ	25
B	Tái lập bảng và hình	26

Danh mục hình

2.1	Phân bố lớp của Covertypes	5
4.1	Các tầng đầu của Decision Tree Covertypes cơ sở	9
4.2	Confusion matrix của Decision Tree Covertypes cơ sở	11
4.3	Feature importance của Decision Tree Covertypes cơ sở	13
5.1	Macro-F1 và độ phức tạp cây trên Covertypes	16
5.2	Train-test gap sau regularization	17

Danh mục bảng

2.1	Đặc trưng của ba bộ dữ liệu	3
4.1	Decision Tree cơ sở trên ba bộ dữ liệu	10
4.2	Recall theo lớp của Covertypes	12
5.1	Tham số cải tiến được chọn bằng cross-validation	15
5.2	So sánh E0–E4 trên Covertypes	15
6.1	Tác động của E4 trên ba bộ dữ liệu	18
6.2	Mô hình tốt nhất trên từng bộ dữ liệu	19
6.3	So sánh custom tree và sklearn	20
A.1	Benchmark đầy đủ: 3 datasets $\times$ 4 models	25
B.1	Nguồn tái lập bảng và hình	26

Chương 1

Giới thiệu

1.1 Bối cảnh và động lực

Decision Tree là mô hình học máy phổ biến nhờ khả năng diễn giải trực quan, không yêu cầu chuẩn hóa đặc trưng và có thể biểu diễn quan hệ phi tuyến bằng các luật *if-then* [3]. Tuy nhiên, phép chia tham lam có thể làm cây ghi nhớ tập huấn luyện, phát triển quá sâu và tạo nhiều lá chỉ chứa rất ít mẫu. Khi đó, mô hình vẫn giải thích được một dự đoán riêng lẻ nhưng khó diễn giải ở mức toàn cục và có nguy cơ phương sai cao.

Vấn đề trung tâm của nghiên cứu là: *làm thế nào xây dựng, phân tích và cải tiến một Decision Tree sao cho cây vẫn dự đoán tốt, tổng quát hóa tốt hơn và giữ được cấu trúc đủ gọn để diễn giải?*

1.2 Phạm vi cốt lõi và phần mở rộng

Nghiên cứu được tổ chức thành hai tầng để bám sát yêu cầu Lab trước khi mở rộng:

- **Core Lab:** dùng Covertypes làm case study chính để xây dựng Decision Tree cơ sở, trực quan cây sinh ra, phân tích split và luật quyết định, đo accuracy, error rate, macro-F1, confusion matrix, overfitting và đánh giá ba phương pháp cải tiến.
- **Extended study:** dùng Letter Recognition và Handwritten Digits làm hai bộ dữ liệu kiểm tra độ bền của kết luận; Random Forest, SVM-RBF và KNN chỉ đóng vai trò mô hình đối chứng.

Mặc dù đề bài chỉ yêu cầu một bộ dữ liệu, Covertypes được dùng để thực hiện đầy đủ mọi phân tích bắt buộc. Hai bộ dữ liệu còn lại giúp kiểm tra liệu kết luận về Decision Tree có giữ được trong các chế độ dữ liệu khác hay không, thay vì thay đổi trọng tâm của Lab.

1.3 Công trình liên quan

ID3 đặt nền móng cho việc sinh cây bằng information gain [1]; C4.5 mở rộng xử lý thuộc tính liên tục và dùng gain ratio [2]. Nghiên cứu này dùng CART với split nhị phân, Gini impurity và Cost-Complexity Pruning [3]. Random Forest giảm phương sai bằng ensemble nhiều cây [4]; SVM và KNN đại diện lần lượt cho phương pháp kernel và phương pháp dựa trên lân cận [5,6]. Hierarchical Shrinkage bổ sung một hướng regularization khác: giữ nguyên topology nhưng làm mượt ước lượng dự đoán ở các nút sâu [7].

1.4 Mục tiêu nghiên cứu

Các mục tiêu cốt lõi được đặt trước các mục tiêu mở rộng:

1. xây dựng và đánh giá một Decision Tree cơ sở có cấu hình tái lập;
2. phân tích cây sinh ra thông qua root split, các luật quyết định, độ sâu, số lá và lỗi theo lớp;
3. áp dụng ba chiến lược cải tiến gồm pre-pruning, Cost-Complexity Pruning (CCP) và Hierarchical Shrinkage (HS);
4. so sánh cây cơ sở với từng cây cải tiến theo hiệu năng, overfitting và độ phức tạp;
5. kiểm tra độ bền của kết luận trên ba chế độ dữ liệu;
6. đặt Decision Tree trong tương quan với Random Forest, SVM và KNN mà không suy rộng vượt quá cấu hình đã khảo sát.

1.5 Câu hỏi nghiên cứu

- **RQ1:** Decision Tree thay đổi như thế nào giữa dữ liệu đa lớp, dữ liệu ảnh ít mẫu và dữ liệu bảng quy mô lớn?
- **RQ2:** Decision Tree có ưu thế và hạn chế gì so với Random Forest, SVM và KNN trong các cấu hình đã thử nghiệm?
- **RQ3:** Pre-pruning, CCP và HS tác động như thế nào đến hiệu năng, overfitting và độ phức tạp của cây?

1.6 Cấu trúc báo cáo

Chương 2 mô tả dữ liệu và vai trò của từng bộ dữ liệu. Chương 3 trình bày CART cơ sở, cấu hình, chỉ số và protocol. Chương 4 phân tích trực tiếp cây Covertypes sinh ra. Chương 5 trình bày ba phương pháp cải tiến và thí nghiệm kết hợp. Chương 6 tổng hợp so sánh cốt lõi, robustness, benchmark, tính tái lập và giới hạn. Chương 7 kết luận và trả lời ba câu hỏi nghiên cứu.

Chương 2

Mô tả dữ liệu

2.1 Thiết kế phạm vi: một case study chính và hai robustness datasets

Covertypes là bộ dữ liệu chính để thực hiện đầy đủ các yêu cầu của Lab. Letter Recognition và Handwritten Digits không thay thế case study chính; chúng được dùng để kiểm tra liệu kết luận có bền trước thay đổi về số lớp, số chiều và số mẫu hay không.

Bảng 2.1: Đặc trưng và vai trò của ba bộ dữ liệu sau bước chuẩn bị.

Bộ dữ liệu	Mẫu	Thuộc tính	Lớp	Train/Test	Vai trò
Letter Recognition	18,668	16	26	14,934 / 3,734	Robustness: 26 lớp
Handwritten Digits	1,797	64	10	1,437 / 360	Robustness: ảnh nhỏ
Covertypes	581,012	54	7	464,809 / 116,203	Primary case study

2.2 Letter Recognition

Letter Recognition của UCI mô tả 26 chữ cái in hoa bằng 16 thuộc tính hình học đã trích xuất, chẳng hạn kích thước hộp bao, vị trí và phân bố điểm ảnh [8]. Target là ký tự từ A đến Z; các feature nhận giá trị nguyên trong khoảng 0–15. Sau khi loại 1,332 dòng trùng hoàn toàn trước khi chia dữ liệu, bộ dữ liệu còn 18,668 mẫu.

Bộ dữ liệu này phù hợp để kiểm tra khả năng phân loại nhiều lớp của một cây đơn. Ranh giới giữa các chữ cái có thể phức tạp, vì vậy kết quả cũng làm rõ lợi ích giảm phương sai của Random Forest và cấu trúc lân cận của KNN.

2.3 Handwritten Digits

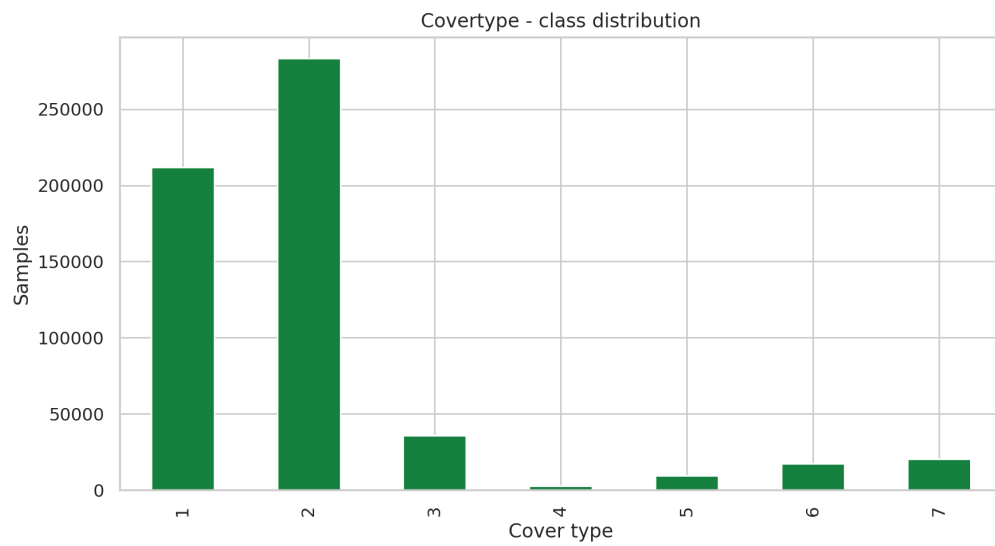
Handwritten Digits gồm 1,797 ảnh chữ số viết tay từ 0 đến 9. Mỗi ảnh 8×8 được biểu diễn bởi 64 mức cường độ điểm ảnh [9]. Target là chữ số; dữ liệu có ít mẫu hơn Letter nhưng số chiều cao hơn và mang cấu trúc không gian của ảnh.

Đây là robustness dataset cho chế độ dữ liệu nhỏ, nhiều chiều. Nó làm bộc lộ hạn chế của các split theo từng trục của Decision Tree và tạo điều kiện phù hợp cho SVM–RBF hoặc KNN.

2.4 Covertypes – case study chính

Covertypes của UCI gồm 581,012 mẫu, 54 thuộc tính và 7 loại lớp phủ rừng [10]. Target **Cover_Type** nhận giá trị từ 1 đến 7. Các feature kết hợp biến địa hình liên tục với biến nhị phân mô tả khu vực hoang dã và loại đất; dữ liệu không có giá trị thiếu trong bản materialized của project.

Covertypes phù hợp nhất với yêu cầu Lab vì có ba đặc điểm: quy mô đủ lớn để cây phát triển sâu, dữ liệu bằng phẳng phù hợp với luật feature–threshold, và phân bố lớp không đều khiến accuracy đơn thuần chưa đủ. Cây cơ sở đạt depth 41 với 23,956 lá, do đó ảnh hưởng của regularization có thể quan sát rõ cả về hiệu năng lẫn cấu trúc.



Hình 2.1: Phân bố 7 lớp trong Covertypes. Chênh lệch số mẫu giữa các lớp là lý do báo cáo macro-F1 và recall theo lớp bên cạnh accuracy.

2.5 Chuẩn bị dữ liệu và chống leakage

Tất cả thí nghiệm dùng `test_size=0.20`, `random_state=42` và chia phân tầng theo nhãn. Letter Recognition dùng đúng train/test đã tạo sau bước loại trùng. `StandardScaler` chỉ được fit trên tập train và chỉ áp dụng cho SVM, KNN; Decision Tree và Random Forest dùng feature gốc. Tập test không tham gia chọn mô hình, α hoặc λ và chỉ được dùng cho báo cáo cuối cùng.

Chương 3

Mô hình Decision Tree cơ sở

3.1 CART xây dựng cây như thế nào

Nghiên cứu dùng CART thông qua `DecisionTreeClassifier` của scikit-learn [3, 11]. Tại một nút chứa tập mẫu S , Gini impurity được tính bởi

$$Gini(S) = 1 - \sum_{k=1}^K p_k^2, \quad (3.1)$$

trong đó p_k là tỷ lệ mẫu thuộc lớp k . Với candidate split dùng feature j và threshold t , mức giảm impurity là

$$\Delta G(j, t) = Gini(S) - \frac{n_L}{n} Gini(S_L) - \frac{n_R}{n} Gini(S_R). \quad (3.2)$$

CART chọn $(j^*, t^*) = \arg \max_{j, t} \Delta G(j, t)$, chia dữ liệu thành hai nút con và lặp lại quy trình. Dạng giả mã rút gọn như sau:

```
BuildTree(S):  
    if Stop(S): return Leaf( $\hat{p}(y | S)$ )  
     $(j^*, t^*) \leftarrow \arg \max_{j, t} \Delta G(j, t)$   
     $S_L, S_R \leftarrow \text{Split}(S, j^*, t^*)$   
    return Node( $j^*, t^*, \text{BuildTree}(S_L), \text{BuildTree}(S_R)$ )
```

Khi điều kiện dừng quá lỏng, cây tiếp tục chia tới các nhánh ít mẫu và có thể đạt lỗi train bằng không. Đây là cơ chế tạo phương sai cao mà ba phương pháp cải tiến ở Chương 5 nhằm tới.

3.2 Cấu hình baseline E0

Baseline E0 là một CART chưa giới hạn độ sâu:

```
DecisionTreeClassifier(  
    criterion="gini",  
    splitter="best",  
    max_depth=None,  
    min_samples_split=2,
```

```

min_samples_leaf=1,
ccp_alpha=0.0,
random_state=42
)

```

Các tham số không liệt kê dùng giá trị mặc định của scikit-learn. E0 chưa sử dụng pre-pruning, CCP hay HS; do đó đây là mốc phù hợp để đo mức overfitting và tác động của từng cải tiến.

3.3 Protocol train, validation và test

Quy trình được cố định trước khi xem kết quả test:

1. làm sạch hoặc materialize dữ liệu;
2. tạo một phép chia train/test 80/20 có stratification với `random_state=42`;
3. dùng 3-fold cross-validation chỉ trên train để chọn pre-pruning, α và λ ;
4. fit lại cấu hình đã chọn trên toàn bộ train;
5. mở test đúng một lần để báo cáo kết quả cuối.

Baseline Covertypes còn được kiểm tra độc lập bằng 5-fold cross-validation để đánh giá độ ổn định. Mọi mô hình trong benchmark dùng cùng test set; scaler chỉ học từ train.

3.4 Chỉ số đánh giá

Accuracy và error rate được định nghĩa bởi

$$Accuracy = \frac{\text{số dự đoán đúng}}{N}, \quad Error\ rate = 1 - Accuracy. \quad (3.3)$$

Với mỗi lớp k , $F1_k = 2P_kR_k/(P_k + R_k)$ và

$$Macro-F1 = \frac{1}{K} \sum_{k=1}^K F1_k. \quad (3.4)$$

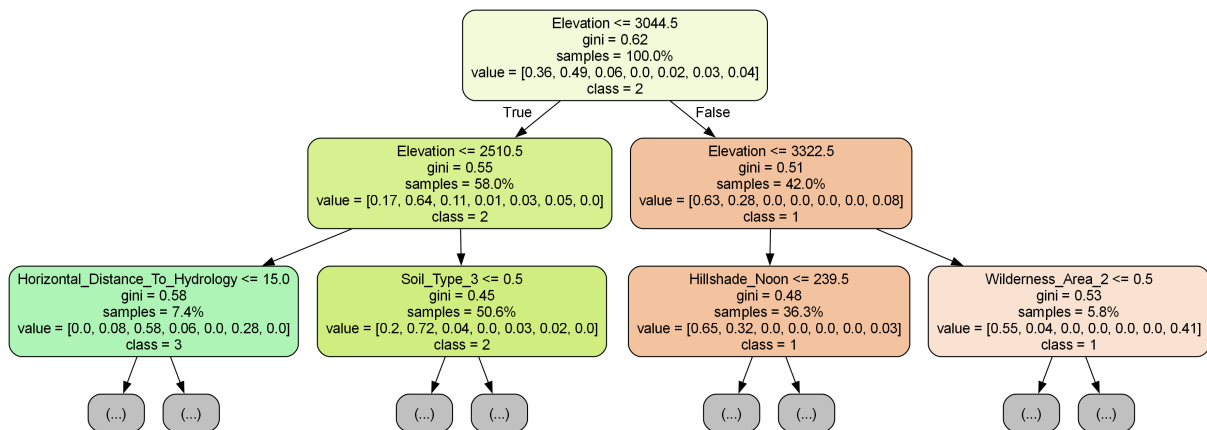
Macro-F1 cho mỗi lớp trọng số ngang nhau nên phù hợp với Covertypes mất cân bằng. Nghiên cứu đồng thời báo cáo confusion matrix, recall theo lớp, train-test gap, thời gian fit/predict, depth và number of leaves. Train-test gap được tính bằng train accuracy trừ test accuracy và được dùng như một tín hiệu thực nghiệm của high variance, không phải một kiểm định thống kê độc lập.

Chương 4

Phân tích cây Decision Tree sinh ra

4.1 Cây Covertypes cơ sở

Hình 4.1 trình bày các tầng đầu của E0. Toàn cây không thể hiển thị trên một trang mà vẫn đọc được vì có depth 41 và 23,956 lá; việc chỉ vẽ các tầng đầu bảo toàn khả năng quan sát root split và các nhánh chính.



Hình 4.1: Các tầng đầu của Decision Tree cơ sở trên Covertypes. Toàn cây có depth 41 và 23,956 lá.

4.2 Root split và các luật quyết định tiêu biểu

Root split là $Elevation \leq 3044.5$. Nút gốc chứa 464,809 mẫu train và có Gini 0.6231. Nhánh trái chứa 269,424 mẫu và tiếp tục tách theo $Elevation \leq 2510.5$; nhánh phải chủ yếu đại diện cho các vùng cao hơn. Việc $Elevation$ xuất hiện ở root phù hợp với trực giác miền: độ cao là yếu tố quan trọng liên quan tới loại lớp phủ rừng. Đây là diễn giải về mô hình, không phải khẳng định quan hệ nhân quả.

Ba đường đi dưới đây được trích trực tiếp từ cây đã lưu. Chúng là các lá thuần trên tập train và được chọn vì có depth tương đối ngắn cùng số mẫu hỗ trợ đủ lớn:

Luật 1: dự đoán lớp 2, hỗ trợ 264 mẫu train:

- $Elevation \leq 3044.5$ và ≤ 2510.5 ;
- $Horizontal_Distance_To_Hydrology > 15$ và ≤ 300.5 ;
- $Wilderness_Area_0 = 1$; $Horizontal_Distance_To_Fire_Points \leq 5122$.

Luật 2: dự đoán lớp 5, hỗ trợ 71 mẫu train:

- $Elevation \leq 3044.5$ và ≤ 2510.5 ;
- $Horizontal_Distance_To_Hydrology > 15$; $Wilderness_Area_0 = 1$;
- $Horizontal_Distance_To_Fire_Points > 5122$;
- $Horizontal_Distance_To_Roadways > 621$.

Luật 3: dự đoán lớp 1, hỗ trợ 298 mẫu train:

- $Elevation > 3044.5$, > 3322.5 và > 3360.5 ;
- $Wilderness_Area_2 = 1$; $Soil_Type_31 = 0$;
- $Horizontal_Distance_To_Fire_Points \leq 403$; $Horizontal_Distance_To_Roadways > 2934.5$.

Các luật cho thấy ưu điểm giải thích cục bộ: một dự đoán có thể được truy vết thành chuỗi điều kiện. Tuy nhiên, gần 24 nghìn lá làm việc mô tả toàn cục trở nên không thực tế; đây là động lực trực tiếp cho pruning.

4.3 Hiệu năng baseline và error rate

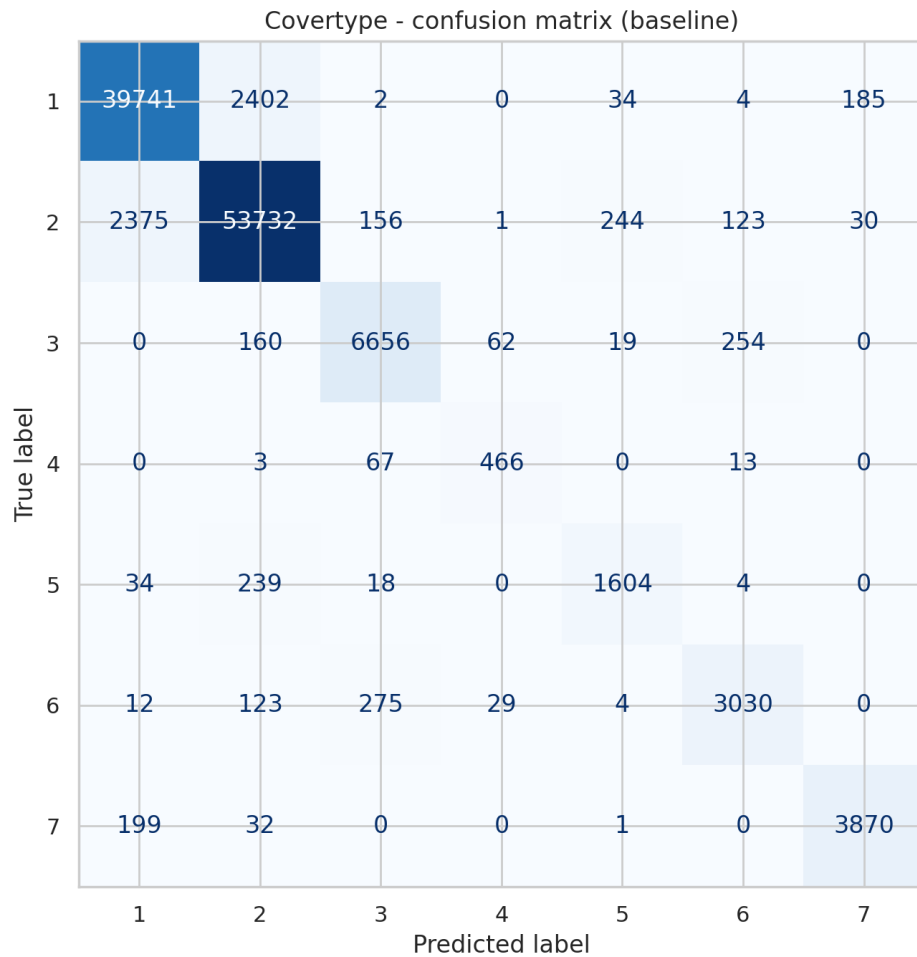
Bảng 4.1: Kết quả Decision Tree cơ sở trên ba bộ dữ liệu.

Bộ dữ liệu	Accuracy	Error rate	Macro-F1
Letter Recognition	86.77%	13.23%	86.77%
Handwritten Digits	82.50%	17.50%	82.30%
Covertypes	93.89%	6.11%	90.34%

Trên Covertypes, E0 đạt train accuracy 100% nhưng test accuracy 93.89%, tạo train-test gap 6.11 điểm phần trăm. Khi đặt cạnh depth 41 và 23,956 lá, lỗi train bằng không là bằng

chứng nhất quán với high variance: cây học rất chi tiết tập train nhưng không giữ được toàn bộ lợi thế đó trên test. Accuracy vẫn cao, vì vậy mục tiêu cải tiến không chỉ là tăng score mà còn phải giảm gap và complexity.

4.4 Confusion matrix và lỗi theo lớp



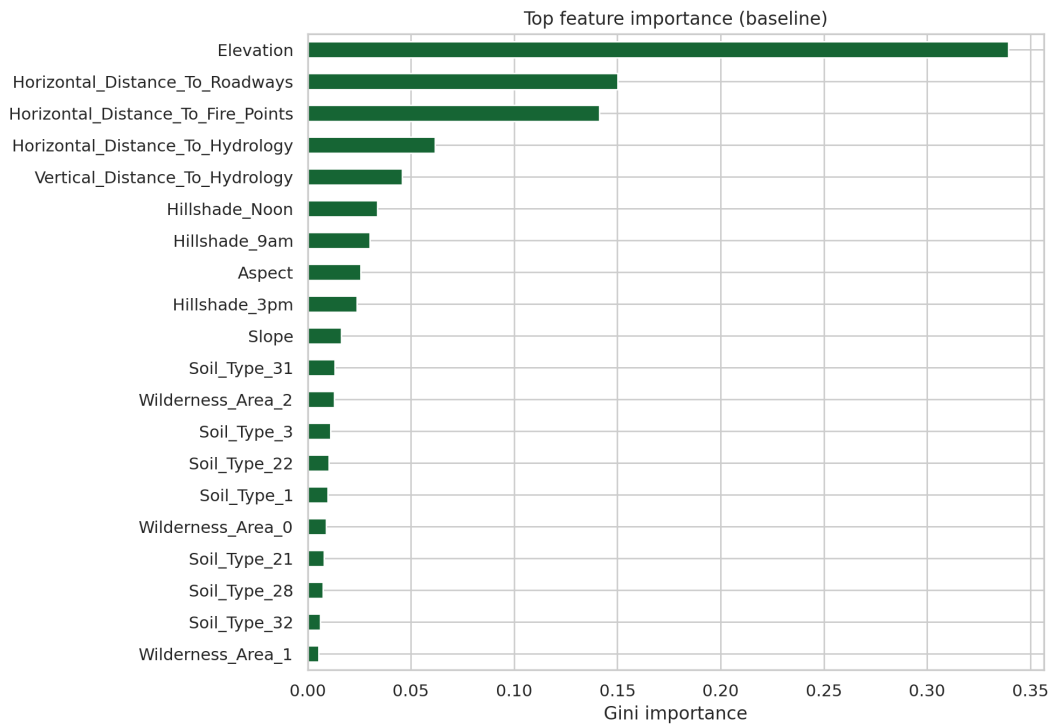
Hình 4.2: Confusion matrix của Decision Tree cơ sở trên 116,203 mẫu test Covertypes.

Bảng 4.2: Recall và số mẫu test của từng lớp Coverttype.

Lớp	Support	Recall	F1
1	42,368	93.80%	93.81%
2	56,661	94.83%	94.81%
3	7,151	93.08%	92.93%
4	549	84.88%	84.19%
5	1,899	84.47%	84.31%
6	3,473	87.24%	87.81%
7	4,102	94.34%	94.54%

Hai lớp lớn 1 và 2 đạt recall gần 94%, trong khi lớp 4 và 5 chỉ khoảng 84–85%. Lỗi lớn nhất tập trung giữa lớp 1–2, lớp 3–6 và các lớp có support nhỏ. Vì vậy, accuracy 93.89% có thể che khuất chất lượng thấp hơn ở lớp thiểu số; macro-F1 90.34% là chỉ số chọn cấu hình phù hợp hơn.

4.5 Feature importance và giới hạn diễn giải



Hình 4.3: Feature importance của Decision Tree cơ sở trên Covertype.

Elevation chiếm importance lớn nhất (0.3395), theo sau bởi Horizontal Distance to Roadways (0.1502), Horizontal Distance to Fire Points (0.1412) và Horizontal Distance to Hydrology (0.0615). Feature importance cung cấp tóm tắt toàn cục về mức sử dụng feature; các path ở phần trước cung cấp giải thích cục bộ cho từng dự đoán. Hai góc nhìn bổ sung lẫn nhau nhưng không biến cây lớn thành một mô hình dễ đọc toàn cục.

Chương 5

Cải tiến Decision Tree

5.1 Ba chiến lược cải tiến

Đề bài yêu cầu từ hai đến ba phương pháp cải tiến. Nghiên cứu đánh giá đúng ba chiến lược nguyên tử: pre-pruning, Cost-Complexity Pruning và Hierarchical Shrinkage. Cấu hình E4 là thí nghiệm kết hợp CCP + HS nhằm kiểm tra tính bổ sung giữa regularization cấu trúc và regularization dự đoán; E4 không được tính là phương pháp bắt buộc thứ tư.

5.1.1 Phương pháp 1 – Pre-pruning

Pre-pruning dừng quá trình sinh cây trước khi các nhánh trở nên quá sâu hoặc quá ít mẫu. Nghiên cứu lựa chọn `max_depth` hoặc `min_samples_leaf` bằng 3-fold CV trên train. Trên Covertypes, cấu hình được chọn là `min_samples_leaf=5`. Cơ chế này giảm phương sai sớm nhưng có thể loại bỏ cấu trúc hữu ích trước khi cây học xong, gây underfitting.

5.1.2 Phương pháp 2 – Cost-Complexity Pruning

CCP fit một cây lớn rồi tìm subtree cân bằng giữa empirical risk và số lá [3, 12]:

$$T_\alpha = \arg \min_{T \subseteq T_0} [R(T) + \alpha |\mathcal{L}(T)|] . \quad (5.1)$$

α càng lớn thì penalty cho số lá càng mạnh. Trên Covertypes, CV chọn $\alpha = 3.442 \times 10^{-6}$. CCP thay đổi topology, vì vậy depth và number of leaves có thể giảm trực tiếp.

5.1.3 Phương pháp 3 – Hierarchical Shrinkage

HS không thay đổi split hoặc topology. Thay vào đó, ước lượng ở nút con được kéo về estimate của nút cha theo hệ số λ [7]:

$$\hat{p}_{HS}(v) = \hat{p}_{HS}(pa(v)) + \frac{n_v}{n_v + \lambda} [\hat{p}(v) - \hat{p}(pa(v))] . \quad (5.2)$$

Nút có nhiều mẫu giữ phần lớn thông tin riêng; nút ít mẫu hoặc λ lớn bị co mạnh hơn về tổ tiên. CV chọn $\lambda = 10$ cho E3 trên cả ba bộ dữ liệu. HS là prediction regularization: depth và số lá phải giữ nguyên sau biến đổi.

5.2 Chọn tham số chỉ trên tập train

Bảng 5.1: Tham số được chọn bằng 3-fold cross-validation trên train.

Dataset	Pre-pruning	CCP	HS trên baseline
Letter	max_depth=16	$\alpha = 0$	$\lambda = 10$
Digits	max_depth=8	$\alpha = 0.002196$	$\lambda = 10$
Covertypes	min_samples_leaf=5	$\alpha = 3.442 \times 10^{-6}$	$\lambda = 10$

Trường hợp $\alpha = 0$ ở Letter cho thấy CV không tìm thấy pruning cấu trúc có lợi trong candidate grid; E2 vì thế trùng E0. Trên Digits, E4 chọn $\lambda = 0$ trên cây CCP, nghĩa là CV không ủng hộ thêm shrinkage sau pruning. Các kết quả âm này được giữ lại để tránh chỉ báo cáo trường hợp thuận lợi.

5.3 Master comparison E0–E4 trên Covertypes

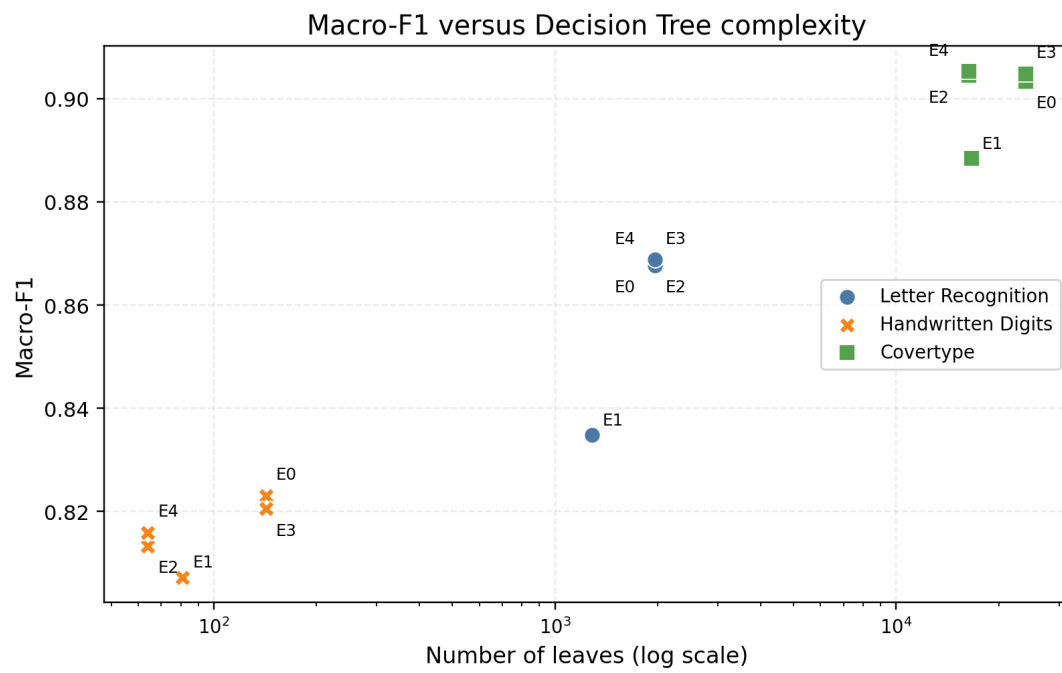
Bảng 5.2: So sánh cây cơ sở và các cấu hình cải tiến trên Covertypes.

ID	Cấu hình	Accuracy	Error	Macro-F1	Gap	Depth	Leaves
E0	Baseline	93.89%	6.11%	90.34%	6.11%	41	23,956
E1	Pre-pruning	92.97%	7.03%	88.85%	4.10%	40	16,637
E2	CCP	93.92%	6.08%	90.45%	5.07%	39	16,361
E3	HS ($\lambda = 10$)	93.97%	6.03%	90.48%	5.70%	41	23,956
E4	CCP + HS	93.93%	6.07%	90.53%	5.04%	39	16,361

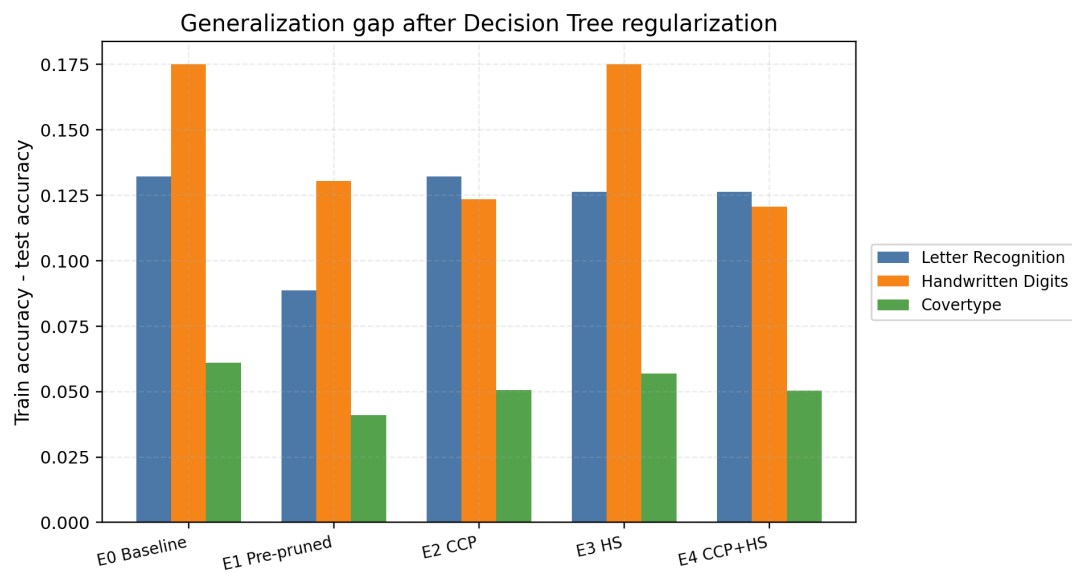
E1 giảm gap và số lá nhưng làm accuracy và macro-F1 giảm nhiều nhất, cho thấy early stopping đã regularize quá mạnh. E2 giảm 31.7% số lá và vẫn tăng nhẹ hai metric test. E3 đạt accuracy cao nhất nhưng giữ nguyên 23,956 lá, đúng với cơ chế prediction regularization. E4 kết hợp được cấu trúc của E2 với dự đoán được shrink, đạt macro-F1 cao nhất.

5.4 Vì sao chọn E4 thay vì E3

E3 cao hơn E4 chỉ 0.04 điểm phần trăm accuracy. Đổi lại, E4 có macro-F1 cao hơn 0.05 điểm phần trăm, ít hơn 7,595 lá và gap thấp hơn 0.66 điểm phần trăm. So với E0, E4 tăng 0.19 điểm phần trăm macro-F1, giảm 31.7% số lá và giảm 17.6% train–test gap. Vì Covertypes mất cân bằng và mục tiêu không chỉ là tối đa accuracy, E4 tạo trade-off kiểu Pareto phù hợp hơn giữa chất lượng dự đoán, generalization, complexity và interpretability.



Hình 5.1: Quan hệ giữa macro-F1 và number of leaves của năm cấu hình trên Covertypes.



Hình 5.2: Train-test gap của các cấu hình cải tiến trên ba bộ dữ liệu.

Chương 6

So sánh và thảo luận

6.1 Cải tiến có khái quát qua ba bộ dữ liệu không?

Để kiểm tra robustness, Bảng 6.1 so sánh cùng một cấu hình kết hợp E4 với baseline. Letter có $\alpha = 0$ nên E4 tương đương HS trên cây chưa prune; Digits chọn $\lambda = 0$ sau CCP; Covertypes dùng cả $\alpha > 0$ và $\lambda = 10$.

Bảng 6.1: Tác động của E4 so với baseline trên ba chế độ dữ liệu.

Dataset	Δ Macro-F1	Δ Leaves	E4 Error	Diễn giải
Letter	+0.11 pp	0.0%	13.12%	HS cải thiện rất nhẹ; CV không chọn pruning
Digits	-0.71 pp	-54.9%	18.33%	Cây nhỏ hơn nhưng mất điểm; mẫu nhỏ làm estimate kém ổn định
Covertypes	+0.19 pp	-31.7%	6.07%	Trade-off rõ nhất giữa score, gap và complexity

Kết quả không ủng hộ phát biểu “HS luôn tăng score”. Lợi ích regularization phụ thuộc số mẫu, độ sâu cây và cấu trúc dữ liệu. Covertypes có đủ dữ liệu và một cây baseline đủ lớn để CCP + HS tạo thay đổi đáng tin cậy về complexity; trên Digits, pruning mạnh làm mất chi tiết hữu ích.

6.2 Benchmark mở rộng: Decision Tree so với ba họ mô hình

Decision Tree là mô hình nghiên cứu chính. Random Forest, SVM-RBF và KNN chỉ là reference baselines đại diện cho ensemble tree, kernel và instance-based learning [4–6]. Bảng 6.2 chỉ tóm tắt mô hình tốt nhất trên từng dataset; bảng đầy đủ nằm ở Phụ lục A.

Bảng 6.2: Mô hình tốt nhất theo test accuracy trong các cấu hình đã khảo sát.

Dataset	Mô hình tốt nhất	Accuracy	Macro-F1
Letter Recognition	Random Forest	95.13%	95.12%
Handwritten Digits	SVM-RBF	97.22%	97.18%
Covertypes	Decision Tree	93.89%	90.34%

Random Forest giảm phương sai hiệu quả trên Letter; SVM-RBF phù hợp với dữ liệu ảnh nhỏ và nhiều chiều; Decision Tree khai thác tốt dữ liệu bảng Covertypes trong cấu hình hiện tại. Kết quả Random Forest Covertypes 79.57% thuộc implementation cuML và một cấu hình không được tìm kiếm siêu tham số toàn diện. Vì vậy, benchmark chỉ mô tả pipeline đã thử nghiệm và không hàm ý Decision Tree luôn vượt Random Forest hoặc SVM.

6.3 Tính tái lập và công bằng thực nghiệm

Các biện pháp kiểm soát gồm:

- một phép chia 80/20 stratified và `random_state=42` dùng chung;
- preprocessing chỉ fit trên train, test không dùng để chọn tham số;
- α và λ được chọn bằng 3-fold CV trên toàn bộ train;
- cùng test set được dùng cho bốn mô hình trên từng dataset;
- baseline Covertypes được kiểm tra thêm bằng 5-fold CV;
- phiên bản thư viện, backend và hardware được lưu cùng artifact.

5-fold CV của E0 Covertypes cho accuracy $93.30\% \pm 0.11$ điểm phần trăm và macro-F1 $89.23\% \pm 0.11$ điểm phần trăm. Độ lệch chuẩn nhỏ cho thấy baseline tương đối ổn định giữa các fold. Ngược lại, một cấu hình đặt trước với `max_depth=20` và `min_samples_leaf=5` chỉ đạt macro-F1 $83.88\% \pm 0.28$, xác nhận regularization phải được chọn bằng validation thay vì áp dụng tùy ý.

6.4 Môi trường tính toán và diễn giải runtime

Decision Tree và HS dùng scikit-learn trên CPU [11]. Random Forest, SVM và KNN trong notebook benchmark dùng RAPIDS cuML trên GPU [13]. Phiên benchmark được cấu hình Kaggle GPU x2 và metadata cuML ghi nhận Tesla T4; các notebook Decision Tree ghi nhận Tesla P100 khả dụng nhưng compute device vẫn là CPU.

Do backend, implementation và thuật toán khác nhau, runtime chỉ phản ánh chi phí thực tế của từng pipeline. Nghiên cứu không diễn giải các số này như speedup CPU-GPU thuần túy và không khẳng định hai GPU được sử dụng song song.

6.5 Threats to validity

1. **Một holdout chính:** ngoài 5-fold CV của baseline Covertypes, kết quả test chính dùng một seed; mức tăng nhỏ cần được lặp lại với nhiều seed.

2. **Tuning chưa exhaustive:** candidate grid tập trung vào Decision Tree improvements; RF, SVM và KNN chưa được tối ưu toàn diện.
3. **Backend không đồng nhất:** Decision Tree chạy CPU còn ba mô hình đối chứng chạy GPU, nên runtime không phải benchmark phần cứng có kiểm soát.
4. **Effect size nhỏ:** chênh lệch score giữa E0, E2, E3 và E4 nhỏ; chưa có kiểm định thống kê nhiều lần lấy mẫu để khẳng định ưu thế tuyệt đối.

Những giới hạn này thu hẹp phạm vi kết luận nhưng không làm mất kết quả chính về complexity: E4 giảm 31.7% số lá trên cùng test protocol mà vẫn giữ macro-F1 gần như không đổi và giảm gap.

6.6 Decision Tree tự cài đặt và đối chiếu với sklearn

Phần này là bonus extension, không thay thế cho các phương pháp cải tiến bắt buộc của core Lab. Mục tiêu là kiểm tra xem các bước cốt lõi của CART có thể được cài đặt lại một cách minh bạch bằng NumPy hay không, đồng thời đối chiếu với `DecisionTreeClassifier` của scikit-learn.

6.6.1 Cài đặt từ đầu và protocol so sánh

`CustomDecisionTreeClassifier` không gọi lại implementation nội bộ của sklearn. Model tự cài đặt tìm split nhị phân bằng cách quét tất cả feature và các ngưỡng liền kề phân biệt, chọn split có Gini impurity nhỏ nhất. Các điều kiện dừng gồm `max_depth`, `min_samples_split` và `min_samples_leaf`; model cũng trả về class probability và văn bản các nhánh quyết định để phục vụ diễn giải.

Hai model được huấn luyện trên cùng bộ Handwritten Digits, cùng split stratified 80/20 `random_state=42`, criterion Gini, `max_depth=8` và `min_samples_leaf=1`. Kết quả được tính trên 360 mẫu test và không dùng test set để chọn tham số.

Bảng 6.3: So sánh custom Decision Tree và sklearn trên Handwritten Digits.

Model	Train acc.	Test acc.	Error	Macro-F1	Depth	Leaves	Fit (s)
Custom tree	93.95%	82.22%	17.78%	82.10%	8	82	0.170
sklearn tree	93.88%	80.83%	19.17%	80.72%	8	81	0.010

Hai model có test prediction trùng nhau 94.72%. Custom tree có accuracy và Macro-F1 cao hơn trong lần chạy này, nhưng không đủ để kết luận rằng implementation tự cài đặt tốt hơn sklearn. Các split có cùng mức giảm Gini có thể được xử lý theo thứ tự khác nhau, dẫn đến topology và prediction khác nhau. sklearn vẫn nhanh hơn rõ ràng nhờ implementation đã tối ưu; kết quả này phân biệt mục tiêu học tập của custom tree với mục tiêu sản xuất. Artifact chi tiết nằm tại `results/custom_vs_sklearn_decision_tree.json` và bảng so sánh tại `results/custom_vs_sklearn_decision_tree__comparison.csv`.

6.6.2 Xác nhận pipeline end-to-end

Để bổ sung kiểm tra khả năng chạy thật, pipeline local được chạy qua đầy đủ các bước *load -> validate/preprocess -> split -> fit -> predict -> metrics*. Letter Recognition được đọc từ split đã làm sạch; Handwritten Digits và Covertypes được đọc từ raw CSV, kiểm tra schema và missing values trước khi chia stratified. Baseline `DecisionTreeClassifier` sau đó được fit và đánh giá trên cả ba dataset.

Lần chạy local hoàn tất thành công cho 18,668 mẫu Letter Recognition, 1,797 mẫu Handwritten Digits và 581,012 mẫu Covertypes; tất cả các bước đều pass trong `results/end_to_end_pipeline_validation.json`. Kết quả test accuracy lần lượt là 86.77%, 82.50% và 93.89%. Đây là bằng chứng cho pipeline CPU của ba dataset trong project. Notebook benchmark bốn model sử dụng RAPIDS/cuML trên GPU vẫn là artifact phụ thuộc môi trường Kaggle và không được coi là một lần xác nhận GPU local.

Chương 7

Kết luận

7.1 Trả lời các câu hỏi nghiên cứu

RQ1 – Decision Tree thay đổi theo chế độ dữ liệu như thế nào? Cây đơn đạt kết quả mạnh nhất trên Covertypes, nơi có nhiều mẫu và feature dạng bảng. Trên Letter và Digits, cây có train accuracy 100% nhưng test accuracy lần lượt chỉ 86.77% và 82.50%, cho thấy phương sai cao rõ hơn khi dữ liệu ít hoặc ranh giới lớp không phù hợp với split theo trục.

RQ2 – Decision Tree mạnh/yếu ra sao so với mô hình đối chứng? Không có mô hình thắng mọi dataset. Random Forest dẫn đầu Letter, SVM–RBF dẫn đầu Digits và Decision Tree dẫn đầu Covertypes trong cấu hình khảo sát. Decision Tree có ưu thế về tốc độ huấn luyện, giải thích theo path và không cần scaling; hạn chế là phương sai cao và complexity có thể tăng rất nhanh.

RQ3 – Ba cải tiến tác động vào đâu? Pre-pruning dừng cây sớm nhưng có nguy cơ underfit; CCP cắt topology và giảm trực tiếp số lá; HS giữ topology nhưng làm mượt prediction ở các nút sâu. Trên Covertypes, E4 CCP + HS có macro-F1 cao nhất, giảm train–test gap khoảng 17.6% và giảm number of leaves khoảng 31.7% so với E0.

7.2 Bài học chính

Một cây tốt hơn không nhất thiết là cây có accuracy cao nhất. Với dữ liệu mất cân bằng và yêu cầu giải thích, lựa chọn cần cân bằng accuracy, error rate, macro-F1, lỗi theo lớp, train–test gap và complexity. E3 có accuracy cao nhất nhưng giữ nguyên 23,956 lá; E4 hy sinh 0.04 điểm phần trăm accuracy so với E3 để đạt macro-F1 cao hơn và một cấu trúc nhỏ hơn nhiều. Đây là lý do E4 được chọn làm cấu hình cuối cho Covertypes.

Kết quả robustness cũng quan trọng: HS/CCP không phải chiến lược tăng điểm phổ quát. Tác động rõ nhất xuất hiện khi baseline đủ sâu, dữ liệu đủ lớn và tham số được chọn bằng train-only validation.

7.3 Hướng phát triển

Nghiên cứu tiếp theo nên lập thí nghiệm qua nhiều seed, dùng nested cross-validation, mở rộng không gian λ và α , đồng thời bổ sung kiểm định thống kê cho effect size nhỏ. Benchmark mô hình đối chứng cần tuning có kiểm soát và backend đồng nhất hơn nếu mục

tiêu chuyển sang so sánh tốc độ. Với Decision Tree, các hướng tiếp theo có thể đo kích thước model, bộ nhớ, độ trễ dự đoán và mức ổn định của từng decision rule trước biến động dữ liệu.

Tài liệu tham khảo

- [1] J. R. Quinlan, “Induction of decision trees,” *Machine Learning*, vol. 1, pp. 81–106, 1986. doi: [10.1007/BF00116251](https://doi.org/10.1007/BF00116251).
- [2] J. R. Quinlan, *C4.5: Programs for Machine Learning*. Morgan Kaufmann, 1993.
- [3] L. Breiman, J. H. Friedman, R. A. Olshen, and C. J. Stone, *Classification and Regression Trees*. Wadsworth International Group, 1984.
- [4] L. Breiman, “Random Forests,” *Machine Learning*, vol. 45, pp. 5–32, 2001. doi: [10.1023/A:1010933404324](https://doi.org/10.1023/A:1010933404324).
- [5] C. Cortes and V. Vapnik, “Support-vector networks,” *Machine Learning*, vol. 20, pp. 273–297, 1995. doi: [10.1007/BF00994018](https://doi.org/10.1007/BF00994018).
- [6] T. M. Cover and P. E. Hart, “Nearest neighbor pattern classification,” *IEEE Transactions on Information Theory*, vol. 13, no. 1, pp. 21–27, 1967. doi: [10.1109/TIT.1967.1053964](https://doi.org/10.1109/TIT.1967.1053964).
- [7] A. Agarwal, Y. S. Tan, O. Ronen, C. Singh, and B. Yu, “Hierarchical Shrinkage: Improving the accuracy and interpretability of tree-based methods,” *arXiv preprint arXiv:2202.00858*, 2022. Available: <https://arxiv.org/abs/2202.00858>.
- [8] D. Slate, “Letter Recognition [Dataset],” UCI Machine Learning Repository, 1991. doi: [10.24432/C5ZP40](https://doi.org/10.24432/C5ZP40).
- [9] scikit-learn developers, “load_digits: Digits classification dataset,” *scikit-learn Documentation*. Available: https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_digits.html. Accessed: Aug. 27, 2026.
- [10] J. Blackard, “Covertypes [Dataset],” UCI Machine Learning Repository, 1998. doi: [10.24432/C50K5N](https://doi.org/10.24432/C50K5N).
- [11] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. Available: <https://jmlr.org/papers/v12/pedregosa11a.html>.
- [12] scikit-learn developers, “Decision Trees and Minimal Cost-Complexity Pruning,” *scikit-learn User Guide*. Available: <https://scikit-learn.org/stable/modules/tree.html>. Accessed: Aug. 30, 2026.
- [13] RAPIDS Development Team, “cuML: GPU Machine Learning Algorithms,” *RAPIDS Documentation*. Available: <https://docs.rapids.ai/api/cuml/stable/>. Accessed: Aug. 27, 2026.

Phụ lục A

Bảng benchmark đầy đủ

Bảng A.1: Kết quả 3 datasets $\times$ 4 models. Runtime là tổng fit và predict của pipeline đã báo cáo.

Dataset	Mô hình	Accuracy	Error	Macro-F1	Runtime (s)
Letter	Decision Tree	86.77%	13.23%	86.77%	0.084
	Random Forest	95.13%	4.87%	95.12%	3.097
	SVM-RBF	92.31%	7.69%	92.32%	3.707
	KNN	94.08%	5.92%	94.08%	0.005
Digits	Decision Tree	82.50%	17.50%	82.30%	0.021
	Random Forest	96.94%	3.06%	96.89%	0.628
	SVM-RBF	97.22%	2.78%	97.18%	0.718
	KNN	96.39%	3.61%	96.34%	0.003
Covertypes	Decision Tree	93.89%	6.11%	90.34%	7.409
	Random Forest	79.57%	20.43%	65.86%	2.995
	SVM-RBF	79.07%	20.93%	63.78%	125.547
	KNN	92.84%	7.16%	87.32%	5.756

Các số runtime không dùng để suy ra speedup CPU-GPU vì backend và implementation khác nhau.

Phụ lục B

Tái lập bảng và hình

Bảng B.1: Nguồn tái lập các kết quả chính.

Kết quả	Notebook	Artifact
Baseline Covertypes E0–E4 và CV	Notebook 03 Covertypes Notebook 04 HS benchmark	dt_covertypes_scalability.json dt_hierarchical_shrinkage__summary.csv
3 × 4 benchmark	Notebook 05 model benchmark	gpu_three_dataset_four_model_comparison__summary.csv
Tree rules và recall lớp	Baseline model + split chung	dt_covertypes_baseline__decision_rules.csv __per_class_metrics.csv

Notebook có tên “three model” vì ba mô hình GPU là RF, SVM và KNN; Decision Tree CPU vẫn được đo trong cùng notebook, nên bảng kết quả có bốn model.